



ANTEPROYECTO DEL TRABAJO DE FIN DE GRADO

Alumno/a	Claudio María Martínez Velasco				
Titulación:	Ingeniería de Computadores				
Tutor/es:	Guillermo Pérez Trabado, Departamento de Arquitectura de Computadores				
Título	Un orquestador de máquinas virtuales para entornos docentes y de investigación. Prueba de concepto				
Subtítulo <i>(solo si en grupo)</i>					
Título en inglés	A virtual machine orchestrator for teaching and research environments. Proof of concept				
Subtítulo en inglés <i>(solo si en grupo)</i>					
Trabajo en grupo:	☒ Sí	☐	☐ No	☒ X	☐
Otros integrantes del grupo:					

La virtualización constituye uno de los pilares fundamentales de la infraestructura tecnológica actual, tanto en el ámbito académico como en el industrial. Su capacidad para permitir la coexistencia de múltiples entornos independientes sobre un mismo hardware ha abierto la puerta a soluciones de docencia más flexibles, entornos de pruebas seguros y sistemas de investigación escalables.

En este contexto académico surge la necesidad de un orquestador de máquinas virtuales (VMs) que proporcione una capa de abstracción por encima de los hipervisores subyacentes (como VMware, Proxmox u Oracle VirtualBox), con el fin de simplificar la administración para usuarios no expertos en manejo de hipervisores, mejorar la modularidad y ofrecer un marco universal para diferentes aplicaciones. En concreto **el despliegue sobre clusters de hipervisores centralizados en un data center de una gran cantidad de máquinas virtuales para entornos docentes asignadas a un gran número de usuarios distintos.**

Contextualización del problema a resolver. Describir claramente de dónde surge la necesidad de este TFG y el dominio de aplicación. En caso de que el TFG se base en trabajos previos, debe aclararse cuáles son las aportaciones del TFG.

Actualmente, existen soluciones comerciales o de código abierto para la gestión de máquinas virtuales, pero suelen estar ligadas a un hipervisor específico o carecen de la flexibilidad necesaria para adaptarse a escenarios de docencia universitaria. En dichos escenarios, es común la necesidad de desplegar **cientos** de máquinas virtuales organizadas por asignatura, alumno y práctica, lo que introduce una elevada complejidad si se gestiona directamente desde el nivel del hipervisor, con sus herramientas estándar. El precio de las soluciones de orquestación comerciales se ha incrementado de forma importante en los últimos años. Una solución open-source podría ser más económica, aun contemplando costes de instalación, formación y mantenimiento.

La propuesta de este TFG consiste en el desarrollo de un orquestador modular de máquinas virtuales, implementado como un servicio Linux con interfaz REST, que proporcione un modelo de administración unificado basado en **descriptores** (descripciones de infraestructuras virtuales). Este orquestador permitirá definir, organizar y gestionar VMs de manera independiente de la tecnología de virtualización subyacente, incorporando además mecanismos de herencia de configuraciones, control de usuarios (propietarios y derechos de acceso), y despliegue automático de infraestructuras educativas. Permitiendo un manejo sencillo de cientos de máquinas. La idea es ofrecer un interface de usuario más sencillo y simplificado, que permita delegar las tareas de gestión de las máquinas virtuales desplegadas para la docencia, a ciertos usuarios, como los profesores de asignaturas. También a los usuarios finales, como los alumnos, podrían tener cierto control mínimo de sus máquinas virtuales asignadas. Todo esto libera al administrador de los hipervisores de cierto trabajo. También permite el despliegue automático de grupos grandes de máquinas virtuales de una forma automatizada y repetible.

El trabajo se basa en librerías Python creadas anteriormente por Guillermo Pérez. Estas librerías hacen de interfaz con hipervisores VMWare, y también realizan un despliegue de una infraestructura virtual descrita en un fichero YAML. Asimismo, utiliza un IPAM externo para la gestión de direcciones IP en el despliegue de la arquitectura virtual.

En el presente TFG, se amplía esto creando un orquestador que maneja muchas máquinas virtuales, manteniendo la información persistente y coordinada con los hipervisores, así como lo mencionado anteriormente. Se adaptará el módulo de despliegue de infraestructuras al orquestador. Será ampliable a otros hipervisores aparte de VMWare al estar diseñado de forma agnóstica respecto al hipervisor subyacente. Por la forma de organizar las máquinas virtuales, será de utilidad para otras aplicaciones, aparte de la docencia, si bien, el primer objetivo es servir para entornos docentes.

Descripción detallada de en qué consistirá el TFG. En caso de que el objeto principal del TFG sea el desarrollo de software, además de los objetivos generales deben describirse sus funcionalidades a alto nivel.

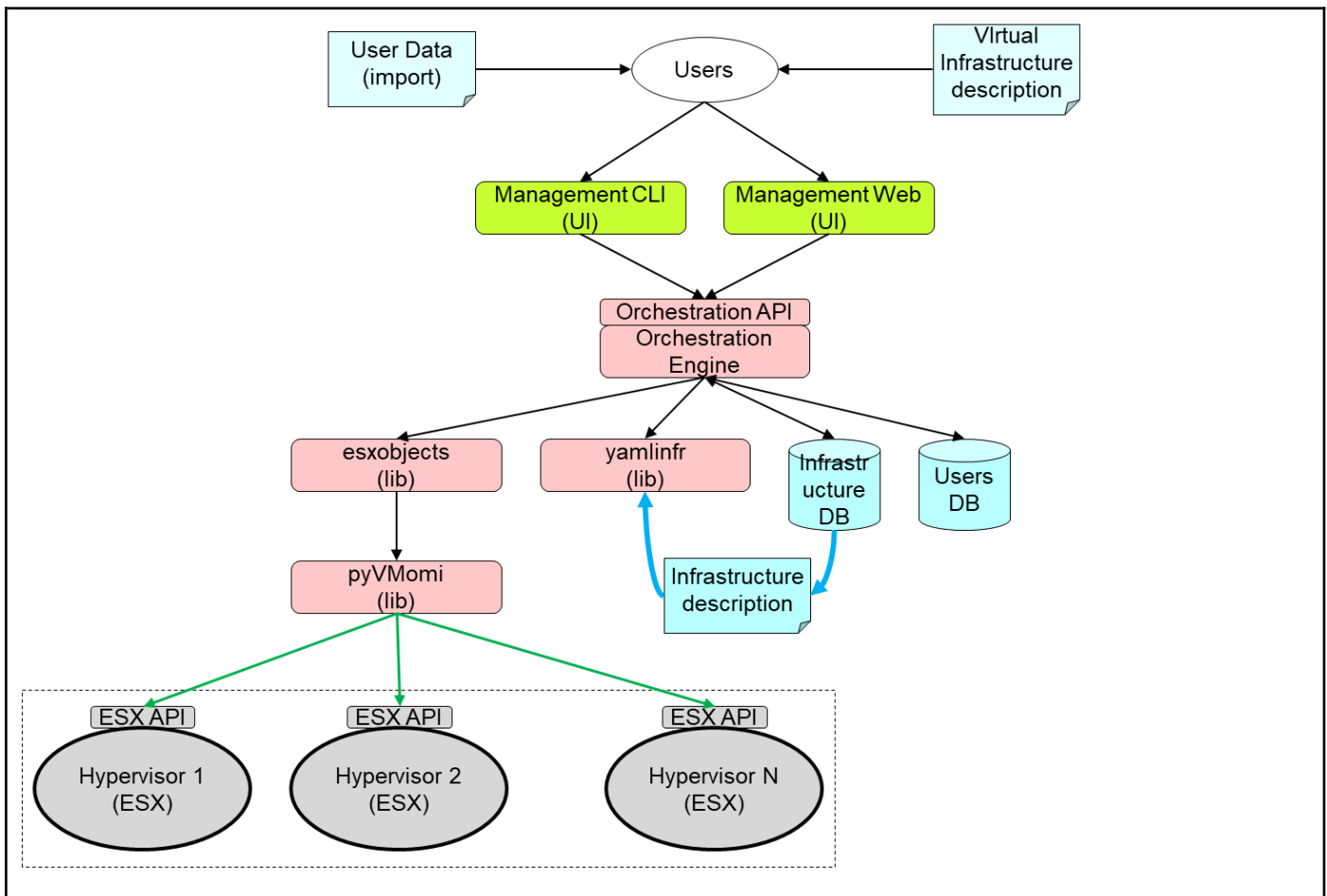
El **objetivo general** del TFG es diseñar e implementar un **orquestador de máquinas virtuales multiplataforma**, con una arquitectura modular y extensible, que pueda ser utilizado tanto en entornos docentes como de investigación.

Dada la envergadura de este diseño, el presente TFG no pretende abarcar el desarrollo completo del sistema, ya que eso sobrepasa con mucho las horas de dedicación necesarias. Se propone una prueba de concepto, donde se deje una arquitectura con cierta funcionalidad, para que sea ampliada posteriormente.

Objetivos específicos:

1. **Diseñar un modelo de descriptores de VMs** que represente de forma unificada tanto la definición como el ciclo de vida de una máquina virtual, incluyendo estados como *Provisioned*, *Created*, *Running*, *Paused*, *Stopped* y *Failed*.
2. **Implementar un servicio Linux** que mantenga un registro centralizado de todos los descriptores, actuando como capa de abstracción respecto a los hipervisores reales.
3. **Definir un sistema jerárquico en árbol** para organizar las VMs, permitiendo la herencia en cascada de configuraciones comunes y la aplicación de permisos y políticas de acceso en distintos niveles.
4. **Desarrollar conectores de comunicación para distintos hipervisores**, empezando con un hipervisor *mock* de pruebas y posteriormente extendiendo la compatibilidad a VMware. Y como desarrollo futuro, otros como Proxmox.
5. **Incorporar un sistema de control de usuarios y roles** (superusuario, manager y usuario simple) con permisos heredables en la jerarquía de carpetas.
6. **Diseñar un módulo especializado para despliegues docentes**, capaz de crear infraestructuras completas a partir de listados de alumnos y definiciones de asignaturas, integrándose con un sistema de gestión de direcciones IP (IPAM).
7. **Implementar una interfaz REST y una interfaz web** (UI) que permitan el acceso remoto y simplificado a las funcionalidades del orquestador. La interfaz web, en este TFG será un primer boceto sin pretensión de ser completa.
8. **Garantizar la coherencia de datos** entre las definiciones de VMs, los descriptores y las máquinas reales, estableciendo políticas claras de actualización y sincronización.

IMPORTANTE: Dada la envergadura de un posible orquestador completo, de los citados objetivos, en el presente TFG se desarrollarán funcionalidades parciales, dejando para un desarrollo futuro, la ampliación a funciones más complejas.



Listado de resultados que generará el TFG (aplicaciones, estudios, manuales, etc.)

- Documentación describiendo la arquitectura del orquestador, el modelo de descriptores y las interacciones entre módulos.
- Implementación de un módulo orquestador como servicio Linux con API REST (basado en FastAPI).
- Implementación de un driver de hipervisor mock y, posteriormente, drivers para VMware.
- Módulo de despliegue automático de asignaturas a partir de definiciones YAML.
- Interfaz de usuario web (UI) desarrollada con algún framework de Python.
- Sistema de gestión de roles y permisos basado en jerarquía de carpetas.
- Repositorio GIT público con el código fuente.

METODOLOGÍA:

Descripción de la metodología empleada en el desarrollo del TFG. Especificar cómo se va a desarrollar. Concretar si se trata de alguna metodología existente y, en caso contrario, describir y justificar adecuadamente los métodos que se aplicarán.

El desarrollo se llevará a cabo siguiendo una metodología ágil basada en **SCRUM**, con iteraciones cortas (sprints) de una o dos semanas. Cada sprint incluirá planificación, desarrollo, revisión y documentación de resultados.

FASES DE TRABAJO:

Enumeración y breve descripción de las fases de trabajo en las que consistirá el TFG.

1. Análisis y diseño inicial: Definición del modelo de descriptores, jerarquía en árbol y estados de ciclo de vida.
2. Implementación del hipervisor mock para pruebas independientes de hardware.
3. Desarrollo inicial del módulo orquestador: gestión básica de descriptores, almacenamiento en árbol y operaciones fundamentales.
4. Extensión del orquestador: herencia de configuraciones, control de usuarios y permisos.
5. Exposición del orquestador como servicio Linux: API REST con FastAPI y cliente CLI.
6. Integración de drivers para VMware.
7. Implementación del módulo de despliegue docente, con integración con un IPAM.
8. Diseño de la interfaz web (NiceGUI) para interacción remota.
9. Pruebas, validación y documentación final.

TEMPORIZACIÓN:

La siguiente tabla deberá contener una fila por cada una de las fases enumeradas en la sección anterior. En caso de tratarse de un trabajo en grupo, se añadirá una columna HORAS por cada miembro del equipo. Debe especificarse claramente el número de horas dedicado por cada alumno/a y la suma de horas individual deberá ser también de 296.

FASE	HORAS
	<i>Claudio Martínez</i>
1. Análisis y diseño inicial	25
2. Implementación hipervisor mock	20
3. Desarrollo básico del orquestador	35
4. Extensión con herencia y permisos	45
5. API REST y servicio Linux	35
6. Drivers para VMware y Proxmox	40
7. Módulo de despliegue docente	40
8. Interfaz web	20
9. Pruebas y validación	16
10. Documentación y memoria	20
	296

(*) La documentación se irá realizando en cada fase.

TECNOLOGÍAS EMPLEADAS:

Enumeración de las tecnologías utilizadas (lenguajes de programación, frameworks, sistemas gestores de bases de datos, etc.) en el desarrollo del TFG.

Sistema Operativo Linux, y Linux Subsystem for Windows

Lenguajes de programación: Python 3, YAML/JSON.

Frameworks y librerías: FastAPI (REST), Pydantic, Textual, Box, NiceGUI (UI), UML (Visual Paradigm o similar).

Herramientas de virtualización: VMware ESXi, Proxmox VE, hipervisor mock.

Bases de datos y almacenamiento: SQLite o sistema de ficheros jerárquico.

Gestión de configuración y despliegue: Ansible, IPAM.

RECURSOS SOFTWARE Y HARDWARE:

Servidores Linux con VMware ESXi y/o Proxmox.

Máquinas virtuales para pruebas en entorno de desarrollo.

Sistema IPAM para gestión de direcciones IP.

Repositorios GitHub/GitLab.

Herramientas de documentación: LaTeX/LibreOffice, Visio o PlantUML.

Listado de dispositivos (placas de desarrollo, microcontroladores, procesadores, sensores, robots, etc.) o software (IDE, editores, etc.) empleados en el desarrollo del TFG.

Visual Studio CODE / Python

Máquinas virtuales con Linux ligeros.

VMWare Workstation y VMWare ESX para ejecutar las máquinas virtuales.

GitLab o GitHub para gestionar el repositorio GIT del proyecto.

Editor para UML (posiblemente MS Visio).

Herramientas de Inteligencia artificial: MS Copilot, Google Gemini, ChatGPT

Listado de referencias (libros, páginas web, etc.)

VMware. "vSphere Virtual Machine Administration Guide." <https://docs.vmware.com>

Proxmox Virtual Environment Documentation. <https://pve.proxmox.com/pve-docs>

FastAPI Documentation. <https://fastapi.tiangolo.com>

HTTP client <https://www.python-httpx.org/>

NiceGUI Documentation. <https://nicegui.io>

YAML parse, <https://yaml.dev/doc/ruamel.yaml/>

Authentication <https://docs.authlib.org/en/latest/client/api.html>

IPAM <https://phpipam.net/documents/>

Python <https://www.python.org/doc/>

Rich API <https://rich.readthedocs.io/en/latest/>

Málaga, 10 de Diciembre de 2025

Firma tutor/tutora:

Firma cotutor/a:

Firma tutor/a coordinador/a: